JavaScript Array Method - pop()

The `pop()` method is one of the most commonly used array methods in JavaScript. It is used to **remove the last element** from an array.

Table of Contents

What is pop()?

Definition

The `pop()` method removes the **last element** from an array.

It modifies the **original array** and returns the **removed element**.

Why Use pop()?

We use `pop()` when we need to:

- Remove the last item
- Delete the latest notification
- Remove the last task
- Implement Undo functionality
- Remove the most recently added product

Syntax

```
array.pop();
```

Parameters

`pop()` does **not** accept any parameters.

```
array.pop();
```

Return Value

`pop()` returns the **removed element**.

Example

```
const fruits = ["Apple", "Mango", "Orange"];

let removed = fruits.pop();

console.log(removed);
```

OUTPUT

```
Orange
```

How pop() Works

Before

```
["Apple","Mango","Orange"]
```

↓

```
pop()
```

↓

Removed

```
Orange
```

↓

After

```
["Apple","Mango"]
```

Example 1 (Basic)

```
const fruits = [
    "Apple",
    "Mango",
    "Orange"
];
fruits.pop();
console.log(fruits);
```

OUTPUT

```
["Apple","Mango"]
```

EXPLANATION

- `pop()` removes the last element.
- `"Orange"` is removed.
- The original array is modified.

Example 2 (Store Removed

Element)

```
const fruits = [
    "Apple",
    "Mango",
    "Orange"
];
const removedFruit = fruits.pop();
console.log(removedFruit);
console.log(fruits);
```

OUTPUT

```
Orange
["Apple","Mango"]
```

Example 3 (Numbers)

```
const numbers = [
    10,
    20,
    30,
    40
];
numbers.pop();
console.log(numbers);
```

OUTPUT

```
[10,20,30]
```

Example 4 (Objects)

```
const users = [
    {
        name: "John"
    },
    {
        name: "Sara"
    }
];
const removedUser = users.pop();
console.log(removedUser);
console.log(users);
```

OUTPUT

```
{ name: "Sara" }

[
    { name: "John" }
]
```

Example 5 (Empty Array)

```
const numbers = [];

const removed = numbers.pop();

console.log(removed);

console.log(numbers);
```

OUTPUT

```
undefined

[]
```

Example 6 (Multiple pop())

```
const colors = [
    "Red",
    "Blue",
    "Green"
];
colors.pop();
colors.pop();
console.log(colors);
```

OUTPUT

```
["Red"]
```

Example 7 (Inside a Variable)

```
const tasks = [
    "Study",
    "Exercise",
    "Sleep"
];
let lastTask = tasks.pop();
console.log(lastTask);
```

OUTPUT

```
Sleep
```

Real Project Example 1

Shopping Cart

```
const cart = [
    "Laptop",
    "Mouse",
    "Keyboard"
];
cart.pop();
console.log(cart);
```

OUTPUT

```
["Laptop","Mouse"]
```

Real Project Example 2

Todo List

```
const todos = [
    "Study",
    "Exercise",
    "Read Book"
];
const completedTask = todos.pop();
console.log(completedTask);
console.log(todos);
```

OUTPUT

```
Read Book
["Study","Exercise"]
```

Real Project Example 3

Notifications

```
const notifications = [
    "Message",
    "Payment",
    "Order Delivered"
];
notifications.pop();
console.log(notifications);
```

OUTPUT

```
["Message","Payment"]
```

Real Project Example 4

Browser History

```
const history = [
    "Google",
    "YouTube",
    "GitHub"
];
const previousPage = history.pop();
console.log(previousPage);
console.log(history);
```

OUTPUT

```
GitHub
["Google","YouTube"]
```

Real Project Example 5

Undo Feature

```
const actions = [
    "Draw",
    "Erase",
    "Color"
];
const undo = actions.pop();
console.log("Undo:", undo);
console.log(actions);
```

OUTPUT

```
Undo: Color
["Draw","Erase"]
```

Best Practices

✅ Use `pop()` only when removing the **last element**.

✅ Store the removed element if needed.

```
const removed = array.pop();
```

✅ Check if the array is empty before removing.

```
if(array.length > 0){

    array.pop();

}
```

✅ Use meaningful variable names.

```
removedTask

lastProduct

deletedMessage
```

Common Mistakes

Mistake 1

Expecting `pop()` to return the updated array.

Wrong

```
const result = numbers.pop();

console.log(result);
```

Output

```
40
```

It returns the **removed element**, not the array.

Correct

```
numbers.pop();

console.log(numbers);
```

Mistake 2

Passing a parameter.

Wrong

```
numbers.pop(30);
```

`pop()` ignores all parameters.

Correct

```
numbers.pop();
```

Mistake 3

Using `pop()` on an empty array.

```
const arr = [];

console.log(arr.pop());
```

Output

```
undefined
```

Mistake 4

Using the wrong method.

To remove the **first element**, use

```
shift()
```

instead of

```
pop()
```

Interview Questions

What does `pop()` do?

It removes the **last element** from an array.

Does `pop()` modify the original array?

✅ Yes.

What does `pop()` return?

The removed element.

Does `pop()` take any parameters?

❌ No.

What happens if the array is empty?

It returns

```
undefined
```

Which method adds an element at the end?

```
push()
```

Which method removes the first element?

```
shift()
```

Summary Table

Feature	Description
Method	`pop()`
Purpose	Remove the last element
Parameters	None
Returns	Removed element
Changes Original Array	✅ Yes
Removes From	End of the array
Common Uses	Undo, Browser History, Todo List, Shopping Cart

Quick Comparison

Method	Action	Returns	Changes Original Array
`push()`	Add to end	New length	✅ Yes
`pop()`	Remove from end	Removed element	✅ Yes
`unshift()`	Add to beginning	New length	✅ Yes
`shift()`	Remove from beginning	Removed element	✅ Yes

Final Notes

The `pop()` method is one of the fundamental array methods in JavaScript. It is commonly used in:

- ✅ Undo functionality
- ✅ Browser history
- ✅ Shopping carts
- ✅ Todo applications
- ✅ Notifications
- ✅ Stack (LIFO) data structures

Mastering `pop()` is essential for understanding how arrays are modified and for building dynamic JavaScript applications.